

Machine Learning Assignment Week 4

Dataset Header: #13-13-13-1, #13-26-13-1

Stephen Haskins: 21365534

October 25, 2025

Brief Data Visualisation

A brief data visualisation helps reveal patterns, relationships, and potential anomalies that may not be apparent from raw numbers alone. By examining the plots, we can roughly estimate the result we expect to see, making a brief visualisation a good starting point in this assignment. A plot of the both datasets (#13-13-13-1 and #13-26-13-1 respectively) is shown in Figure 1 below. Each point represents a data sample, plotted using the two input features on the x and y axes, coloured according to its class label (± 1)

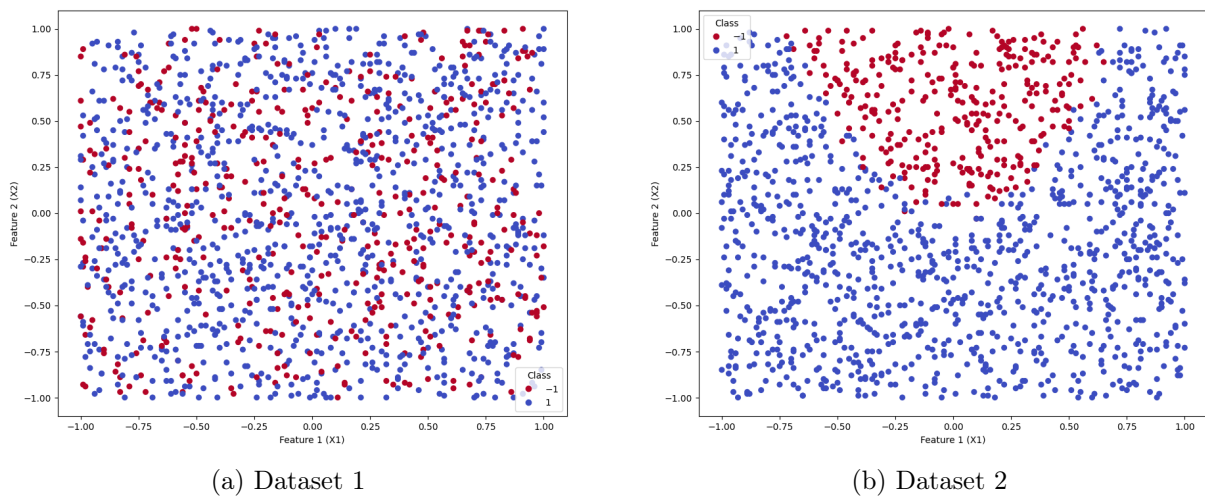


Figure 1: Visualisation of Datasets

Figure 1a shows overly noisy data with no clear separation between classes, and no clear relationship between features. From this, we can expect poor classification performance, as models are likely to struggle to find any meaningful decision boundary.

The second dataset shown in Figure 1b shows a clear separation between the two classes, with minimal overlap. Thus we can expect models to achieve much higher performance with this dataset.

i)

a) Train LR model and perform grid search

In this section we are required to optimise the Logistic Regression classifier by using cross-validation to determine both the maximum polynomial degree and the optimal value of C . These two model parameters are interdependent, thus requiring a grid search across a range of values. Polynomial degrees ranged from 1 to 10, and C values ranged from 1 to 1000. The optimal combination of parameters was selected based on the models prediction accuracy.

5-fold cross-validation was used to estimate the classification accuracy for each combination of parameters, and the mean accuracy across folds was recorded. The accuracy metric was chosen because it directly measures the correct predictions. However, this metric can be misleading

if there is significant class imbalance in the data. Similar to previous assignments, sklearn's LogisticRegression [1] and PolynomialFeatures [2] classes were used. A heat map displaying the mean 5-fold cross-validation accuracy results across the grid search is shown below in Figure 2.

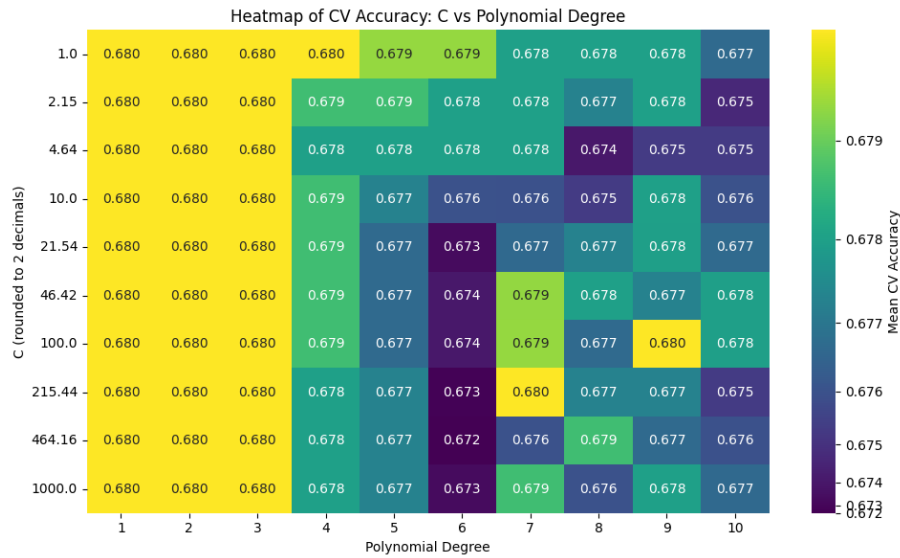


Figure 2: 5-fold cross-validation accuracy heatmap for a range of polynomial degrees and C.

For Dataset 1, the heatmap shows that model performance is relatively insensitive to changes in either C or polynomial degree within the tested range. Mean accuracy remains at 0.68 for all values of C with a polynomial degree ≤ 3 . This suggests that neither increasing model complexity nor adjusting regularisation strength provides meaningful improvement. These results confirm our prediction that the dataset simply contains too much noise, and no meaningful relationship between variables can be identified.

A second plot, visualising the effects of and further refining the parameter C with the optimal polynomial degree (chosen to be 1 for simplicity), is shown in Figure 3

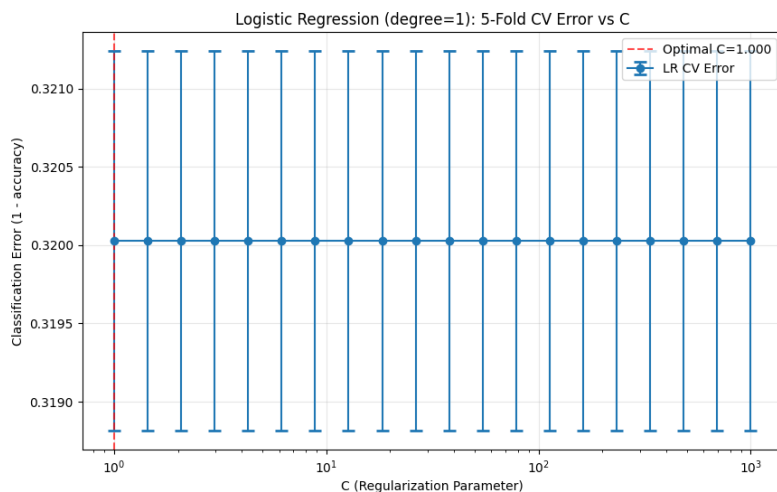


Figure 3: Plot of CV error vs C parameter

The mean cross-validation error remains constant across all tested C values, indicating that the regularisation strength has no influence on the model's performance. The optimal C was taken to be 1.0, but this selection is not statistically significant.

Overall, these cross-validation results suggest that for Dataset 1, a simple linear model (degree = 1, $C=1$) performs as well as more complex alternatives. The insensitivity of accuracy to both parameters confirms that model complexity cannot compensate for the high level of noise in the data.

Finally, Figure 4 shows the decision boundary of the optimal model overlaid on the training data.

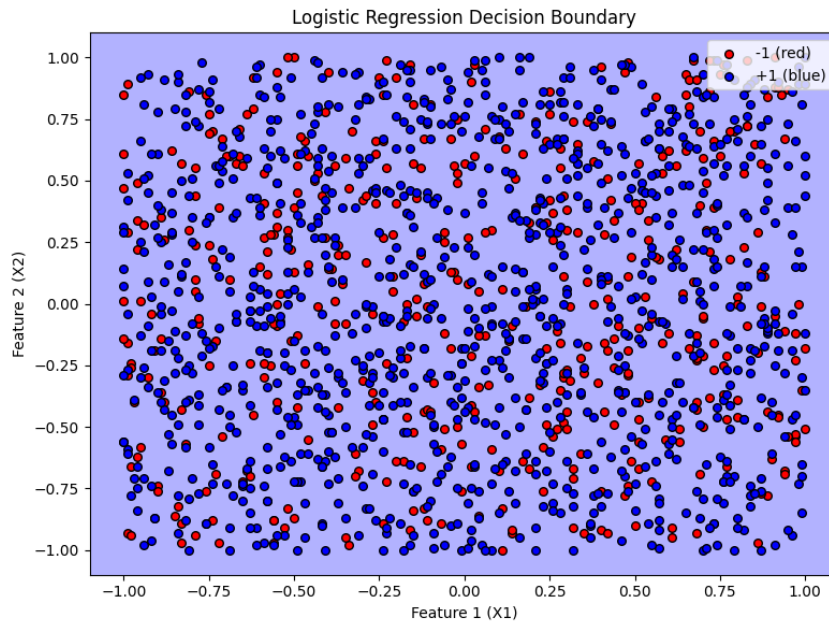


Figure 4: Logistic Regression Decision Boundary

This plot, as well as the following confusion matrix, reveals that the classifier predicts all samples as belonging to the +1 class (blue region), with no separation between classes. This highlights how accuracy can be misleading in the presence of class imbalance, as it reports the trivial majority-class classifier to have an accuracy of 68%, demonstrating that a single metric alone cannot capture true model performance

The outcome indicates that the model has effectively converged to a constant prediction, suggesting that the features provide little discriminative information for the target variable. The absence of a clear boundary is consistent with the cross-validation results, which showed little sensitivity to model complexity. These results confirm that the dataset is too noisy for polynomial Logistic Regression to extract a meaningful boundary.

b) Train a kNN model and find the optimal value of k

The k-Nearest Neighbours (kNN) algorithm is a non-parametric classifier that assigns a class label to a new data point based on the majority class among its k nearest neighbours, measured using Manhattan or Euclidean distance. Unlike Logistic Regression, kNN makes no assumptions about the form of the decision boundary, enabling it to capture nonlinear relationships but making it sensitive to noise and the choice of k .

The model was implemented using sklearn's `KNeighborsClassifier` [3], with Euclidean distance. To identify the optimal value of k , 5-fold cross-validation was performed.

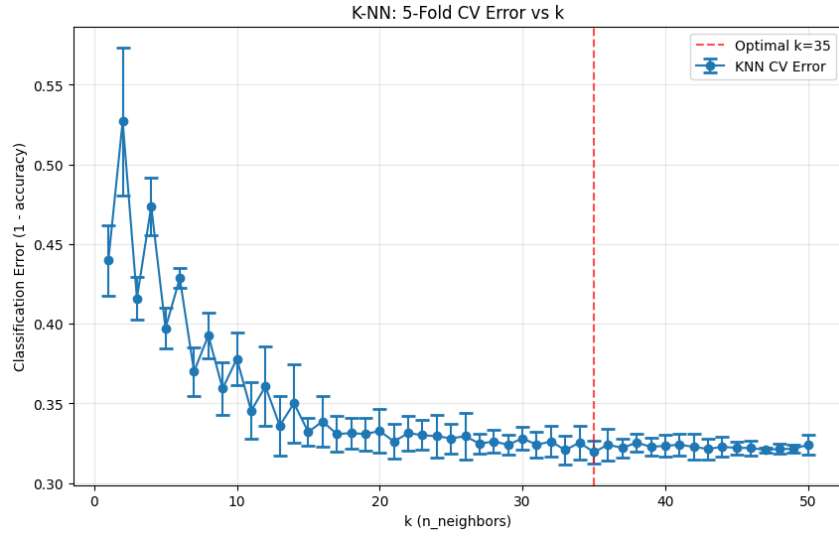


Figure 5: 5-fold CV error vs k

Figure 5 shows the mean CV classification error with error bars representing standard deviation across folds. The error decreases sharply for small k values, then begins to plateau around $k=25$, with a minimum error of 0.319 ± 0.007 at $k=35$. This indicates that larger k values improve generalisation by smoothing local noise, while very small k leads to overfitting.

Figure 6 shows the decision boundary for $k=35$. The classifier forms a few compact clusters of the minority class, reflecting its ability to capture local nonlinear structure. However, the severe overlap between classes limits the achievable accuracy, consistent with the noisy nature of the dataset.

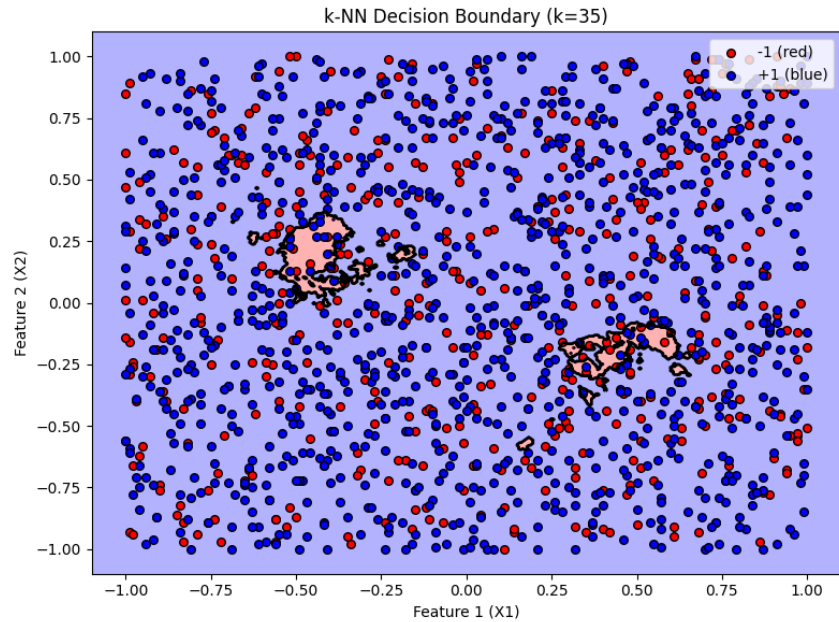


Figure 6: Decision boundary for kNN

Overall, $k=35$ provides the best trade-off between bias and variance. Compared with Logistic Regression, kNN performs slightly better by modelling nonlinear boundaries, though both are constrained by the weak class separability in this data.

c) Plot the confusion matrices for both models

A confusion matrix summarises a classifier's performance by comparing predicted and true class labels. The diagonal entries represent correct predictions, while the off-diagonal entries show misclassifications. Confusion matrices were computed for the Logistic Regression, kNN, and a baseline dummy classifier, which makes random predictions with equal probability for each class. These were generated using sklearn's `confusion_matrix` function [4]. Figure 7 displays the three matrices side by side.

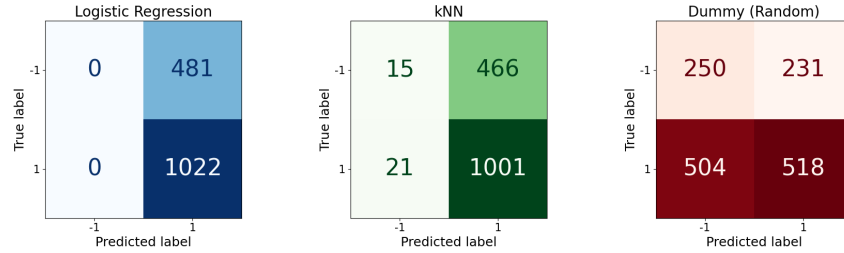


Figure 7: Dataset 1 Confusion Matrices

d) Plot the ROC curves for both models

ROC curves illustrate the trade-off between the true positive rate and the false positive rate across different classification thresholds. To generate the curves for the trained Logistic Regression and kNN classifiers, predicted probabilities for each class were computed for all samples. These probabilities were then used to calculate the true positive and false positive rates needed for plotting the ROC curves.

The area under each curve (AUC) provides a single numerical measure of performance, with higher values indicating better discrimination between classes. Figure 8 shows the resulting ROC curves, with the Logistic Regression and kNN models compared against the random baseline.

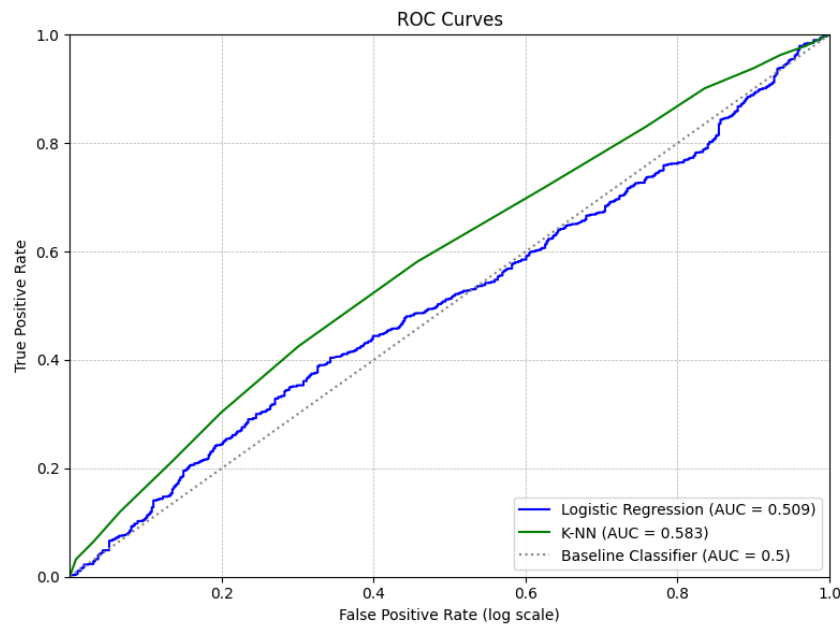


Figure 8: Dataset 1 ROC curves

e) Compare both models

The confusion matrices and ROC curves provide a clear comparison of model performance. The Logistic Regression model predicts all samples as belonging to the positive class (+1), resulting in an AUC of 0.509, close to random guessing. This indicates that the model failed to identify any meaningful decision boundary, consistent with the earlier observation that the dataset contains little separable structure.

The kNN classifier performs moderately better, correctly identifying small subsets of the minority class. Its AUC of 0.583 exceeds both Logistic Regression and the random baseline (AUC=0.5), suggesting limited but genuine discriminative ability. However, the overall improvement remains small, reflecting the noisy nature of the data and the substantial class overlap observed in earlier plots.

The baseline dummy classifier, which predicts classes randomly, achieves roughly balanced outcomes across both classes, as expected. Its confusion matrix and ROC curve confirm chance-level behaviour.

Overall, while the kNN model marginally outperforms Logistic Regression, neither model achieves reliable separation between classes. Given this, no classifier can be confidently recommended for practical use on this dataset. The results indicate that the features provide insufficient information to support effective classification, and collecting higher-quality or more informative data would likely yield greater improvement than further model tuning.

ii) Second Dataset

a) Train LR model and perform grid search

Cross-validation was used to jointly optimise the regularisation parameter C and the maximum polynomial degree. The mean 5-fold accuracy scores across the grid search are shown in a heatmap in Figure 9.

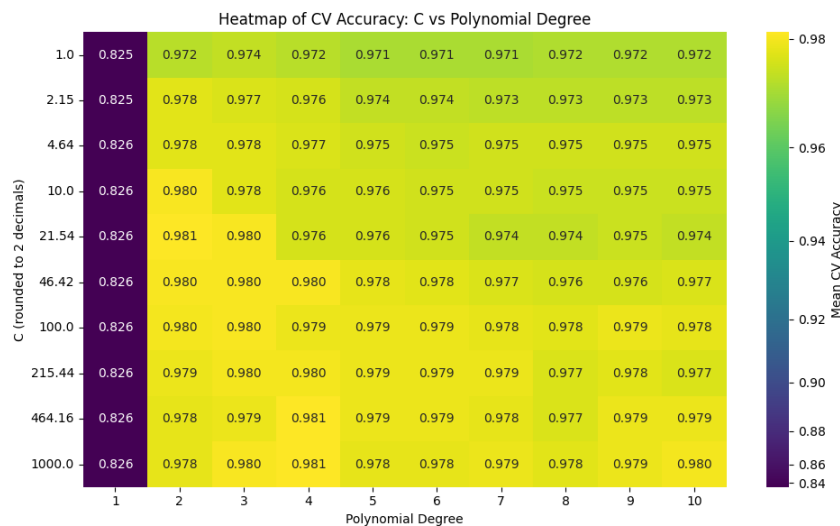


Figure 9: Heatmap showing mean 5-fold cross-validation .

Model performance is consistently high across almost all tested configurations, with mean accuracy values above 0.97. Accuracy increases sharply between degree 1 and degree 2, after which improvements become negligible, indicating that most of the useful nonlinear structure is cap-

tured by the quadratic features. Variation across C values is minimal once $C > 2$, confirming that the model's performance is not strongly dependent on the exact strength of regularisation.

The highest cross-validation accuracy of 0.981 is achieved at degree = 2 and C roughly = 20, though neighbouring settings yield nearly identical results. The heatmap indicates that this dataset contains classes that are highly separable and can well modelled by a low-order polynomial Logistic Regression classifier.

To further refine the choice of C , 5-fold cross-validation was repeated using the optimal polynomial degree (2). The results, shown in Figure 10, display the mean classification error with error bars representing standard deviation across folds.

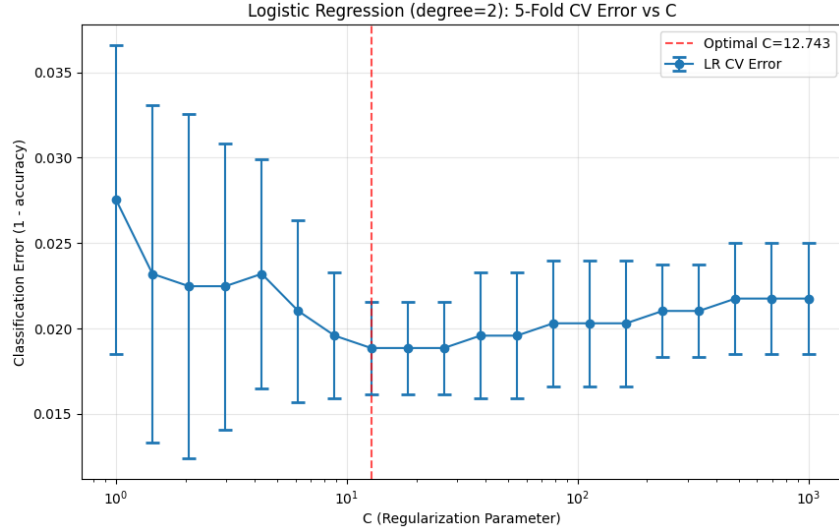


Figure 10: Plot of CV error vs C parameter

The error decreases sharply for small C values and stabilises once $C > 10$, with the minimum error of approximately 0.018 occurring at $C = 11.3$. Beyond this point, performance remains relatively constant with small variability, indicating that the model's generalisation ability is robust to the exact value of C . These results are consistent with the heatmap findings, confirming that moderate regularisation achieves the best balance between bias and variance.

Figure 11 shows the decision boundary of the optimised Logistic Regression model trained with polynomial features. Each point represents a training sample coloured by its true class, while the shaded regions indicate the predicted class.

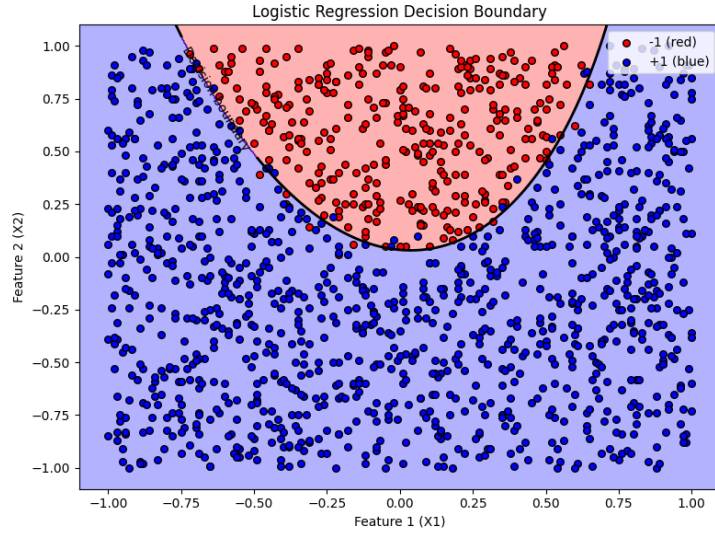


Figure 11: Decision Boundary for Logistic Regression

The model clearly separates the two classes with a smooth, curved boundary, accurately capturing the nonlinear relationship between features. Only a small number of misclassified points appear near the decision boundary, confirming that the quadratic feature expansion was sufficient to model the data's underlying structure. This strong separation is consistent with the high cross-validation accuracy observed earlier, indicating excellent generalisation performance.

b) Train a kNN model and find the optimal value of k

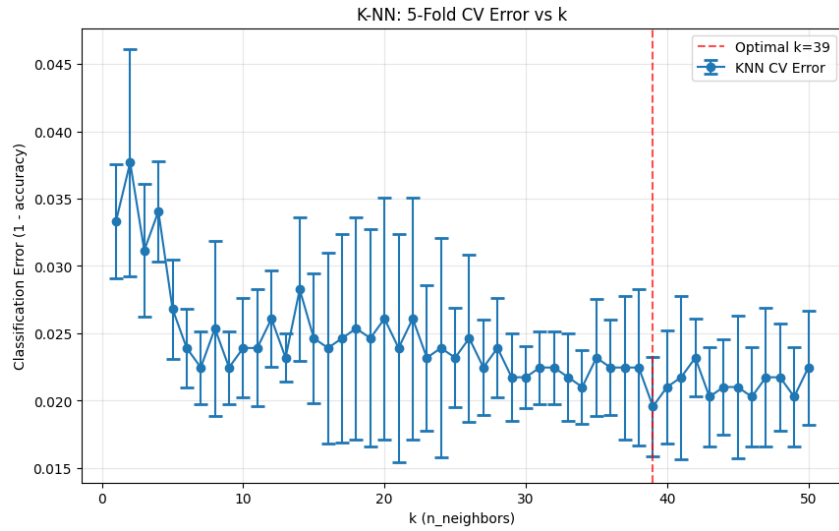


Figure 12: 5-fold CV error vs k

Figure 12 shows the 5-fold cross-validation classification error as a function of the number of neighbours k . The error decreases rapidly for small k values, then stabilises around $k=30$, indicating reduced sensitivity to the exact choice of k . The minimum mean error of approximately 0.020 occurs at $k=39$. Overall, the trend confirms that the dataset is well-structured and can be modelled accurately by a moderately smoothed kNN classifier.

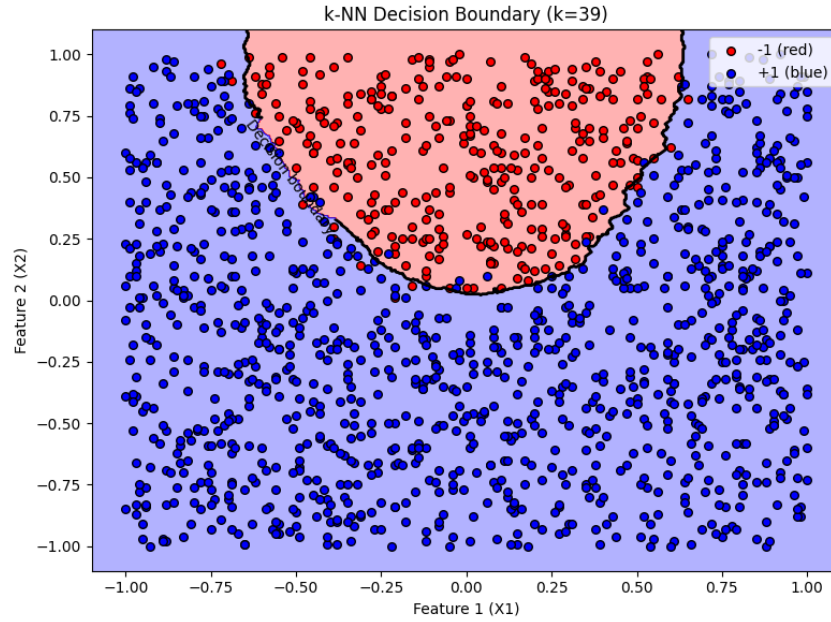


Figure 13: Decision boundary for kNN

Figure 13 shows the decision boundary of the optimised kNN classifier with $k=39$. Each point represents a training sample coloured by its true class, while the shaded regions indicate the predicted class. The boundary closely follows the true class separation, forming a nonlinear contour that accurately captures the curved division between the two clusters. Only a few misclassifications occur near the decision boundary, reflecting the model's sensitivity to local variations but also its strong overall fit. Compared with Logistic Regression, kNN achieves a similarly accurate separation while adapting more flexibly to the data's nonlinear structure.

c) Plot the confusion matrices for both models

Figure 14 displays the confusion matrices for the three models side by side.

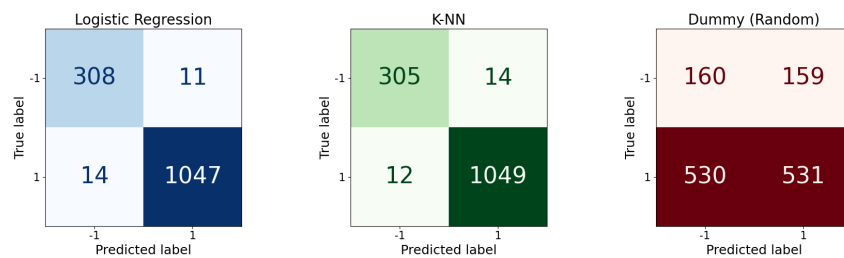


Figure 14: Dataset 2 Confusion Matrix

d) Plot the ROC curves for both models

Figure 15 shows the resulting ROC curves, with the Logistic Regression and kNN models compared against the random baseline. A logarithmic scale was used on the FPR axis to highlight differences in model behaviour at very low FPRs.

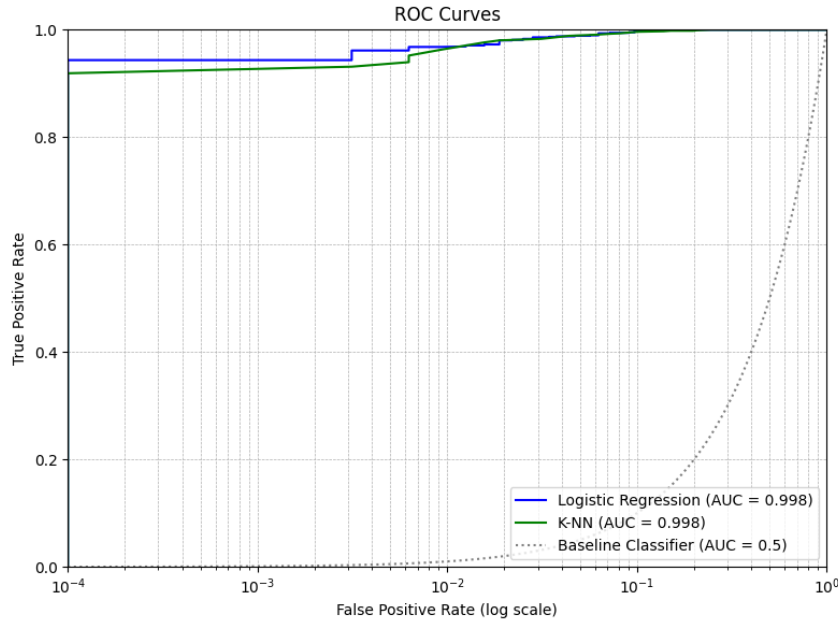


Figure 15: Dataset 2 ROC Curves

e) Compare both models

The confusion matrices (Figure 14) show both trained models achieve near-perfect classification, with only a small number of misclassified samples. Logistic Regression correctly classifies over 98% of instances, while kNN performs comparably with slightly fewer total errors. In contrast, the dummy classifier shows a roughly even distribution of predictions across both classes, reflecting random guessing. These results confirm that both models generalise extremely well on this dataset, consistent with the strong separation visible in earlier plots.

Figure 15 shows the ROC curves. Both trained models achieve near-perfect discrimination, with $AUC = 0.998$ for each, indicating almost complete separation between the two classes. The curves lie close to the top-left corner of the plot, confirming that both models maintain very high true positive rates while keeping false positives close to zero. The baseline classifier follows the logarithmic line ($AUC=0.5$), representing random guessing. These results are consistent with the confusion matrices and decision boundaries, demonstrating that the dataset is highly separable and that both models generalise exceptionally well.

Conclusion

Across both datasets, model performance was dependent on the underlying data quality. For Dataset 1, heavy noise and class overlap prevented meaningful learning, with both Logistic Regression and kNN performing just marginally better than a trivial classifier. In contrast, Dataset 2 exhibited clear separability, allowing both models to achieve near-perfect accuracy and AUC. These results highlight that data structure and feature quality, rather than model complexity, primarily determine classification success.

References

- [1] Sklearn logistic regression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html, 2025.

- [2] Sklearn polynomialfeatures. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html>, 2025.
- [3] Sklearn kneighborsclassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>, 2025.
- [4] Sklearn confusion matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html, 2025.

Appendix: Code

The following is a pdf of the Jupyter Notebook used for this assignment.

clean

October 25, 2025

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (
    classification_report,
    accuracy_score,
    roc_curve,
    roc_auc_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

from matplotlib.colors import ListedColormap
```

```
[2]: # change this directory based on dataset
dataset_x = pd.read_csv('week4_data_2.csv', names=['x1', 'x2', 'y'])

dataset_x.head()
```

```
[2]:      x1    x2  y
0 -0.72  0.28  1
1 -0.34  0.56 -1
2 -0.94 -0.84  1
3  0.18 -0.27  1
4 -0.08  0.17 -1
```

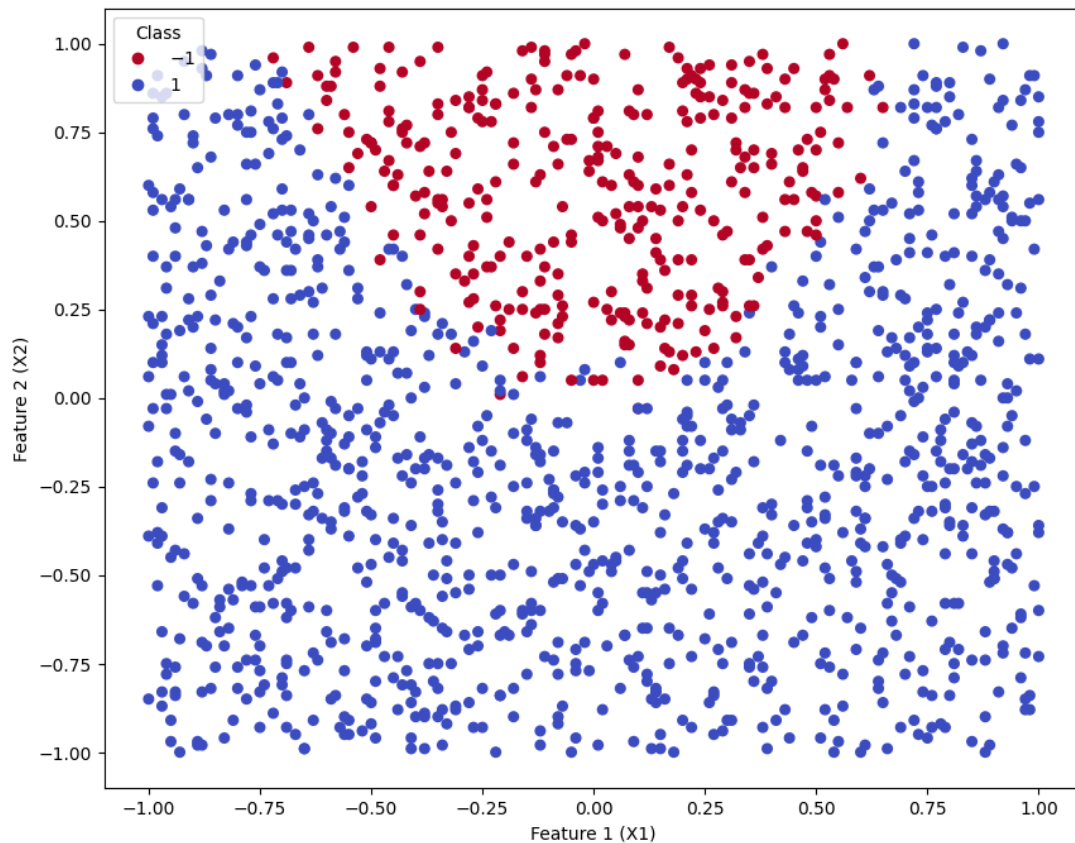
```
[3]: X1 = dataset_x.iloc[:, 0]
      X2 = dataset_x.iloc[:, 1]
      X = np.column_stack((X1, X2))
      y = dataset_x.iloc[:, 2]
```

```
[4]: fig, ax = plt.subplots(figsize=(10, 8))

      scatter = ax.scatter(X1, X2, c=y, cmap='coolwarm_r', marker='o', s=30)

      ax.set_xlabel('Feature 1 (X1)')
      ax.set_ylabel('Feature 2 (X2)')
      ax.legend(*scatter.legend_elements(), title="Class")

      plt.show()
```



```
[5]: # from last assignment
      poly = PolynomialFeatures(degree=5, include_bias=False)
```

```
X_poly = poly.fit_transform(X)

print(f"Original shape: {X.shape}")
print(f"Poly shape: {X_poly.shape}")
print(f"Feature names: {poly.get_feature_names_out(['x', 'y'])}")
```

```
Original shape: (1380, 2)
Poly shape: (1380, 20)
Feature names: ['x' 'y' 'x^2' 'x y' 'y^2' 'x^3' 'x^2 y' 'x y^2' 'y^3' 'x^4' 'x^3
y'
'x^2 y^2' 'x y^3' 'y^4' 'x^5' 'x^4 y' 'x^3 y^2' 'x^2 y^3' 'x y^4' 'y^5']
```

```
[6]: lr = LogisticRegression()
lr.fit(X, y)

y_pred = lr.predict(X)

accuracy = accuracy_score(y, y_pred)
print(f"Accuracy on training data: {accuracy:.4f}")
```

Accuracy on training data: 0.8283

0.1 Grid Search

```
[7]: pipeline = Pipeline(
    steps=[
        ("poly", PolynomialFeatures(include_bias=False)),
        ("logreg", LogisticRegression(penalty="l2")),
    ]
)

# set to 1 for d2, set to 2 for d1
degree_range = list(range(1, 11)) # 1 to 10 inclusive
C_values = np.logspace(0, 3, 10) # from 1 to 1e3

# for grid search
param_grid = {
    "poly__degree": degree_range,
    "logreg__C": C_values,
}
```

```
[8]: # method for cross validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Use accuracy for classification CV selection
grid = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
```

```

        scoring="accuracy",
        cv=cv,
    )

    grid.fit(X, y)

    best_params = grid.best_params_
    best_score = grid.best_score_

    print("Best CV params:", best_params)
    print(f"Best CV mean accuracy: {best_score:.4f}")

    # -----
    # Extract CV results and prepare for plotting
    # -----
    cv_results = pd.DataFrame(grid.cv_results_)

```

Best CV params: {'logreg__C': np.float64(21.544346900318832), 'poly__degree': 2}
 Best CV mean accuracy: 0.9812

```

[9]: cv_results["degree"] = cv_results["param_poly__degree"].astype(int)
    cv_results["C"] = cv_results["param_logreg__C"].astype(float).round(2)

    agg = (
        cv_results.groupby(["C", "degree"], as_index=False)["mean_test_score"]
        .mean()
    )

    heatmap_data = agg.pivot(index="C", columns="degree", values="mean_test_score")
    heatmap_data = heatmap_data.sort_index().sort_index(axis=1)

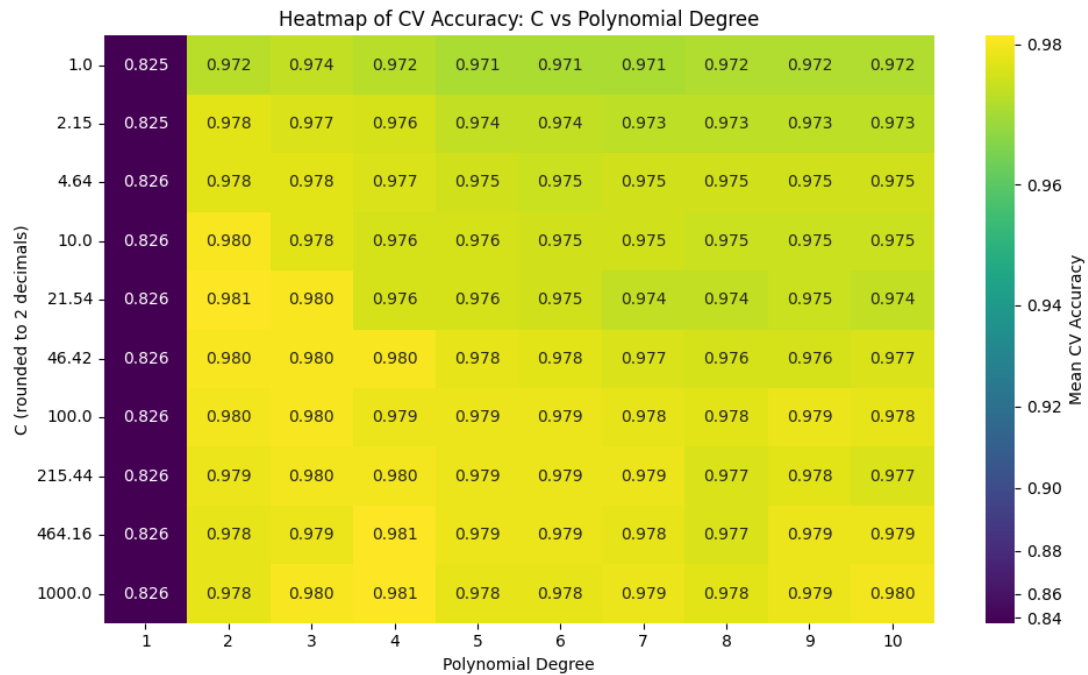
    # enhance detail in the heatmap where most values cluster
    from matplotlib.colors import PowerNorm

    plt.figure(figsize=(10, 6))
    sns.heatmap(
        heatmap_data,
        annot=True,
        fmt=".3f",
        cmap="viridis",
        norm=PowerNorm(gamma=2), # gamma<1 emphasises low values
        cbar_kws={"label": "Mean CV Accuracy"},
    )
    plt.xlabel("Polynomial Degree")
    plt.ylabel("C (rounded to 2 decimals)")
    plt.title("Heatmap of CV Accuracy: C vs Polynomial Degree")
    plt.tight_layout()

```



```
plt.show()
```



0.1.1 Visualisation of C

```
[10]: # best polynomial degree from previous grid search
best_degree = int(best_params["poly__degree"])
print(best_degree)

C_values = np.logspace(0, 3, 20)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# mean and std of classification error (1 - accuracy)
lr_cv_means = []
lr_cv_stds = []

for C in C_values:
    pipeline = Pipeline(
        steps=[
            ("poly", PolynomialFeatures(degree=best_degree,
include_bias=False)),
            ("logreg", LogisticRegression(penalty="l2", C=C, max_iter=10000)),
        ]
    )
```

```

    acc_scores = cross_val_score(pipeline, X, y, cv=cv, scoring="accuracy")
    err_scores = 1.0 - acc_scores # classification error
    lr_cv_means.append(np.mean(err_scores))
    lr_cv_stds.append(np.std(err_scores))

lr_cv_means = np.array(lr_cv_means)
lr_cv_stds = np.array(lr_cv_stds)

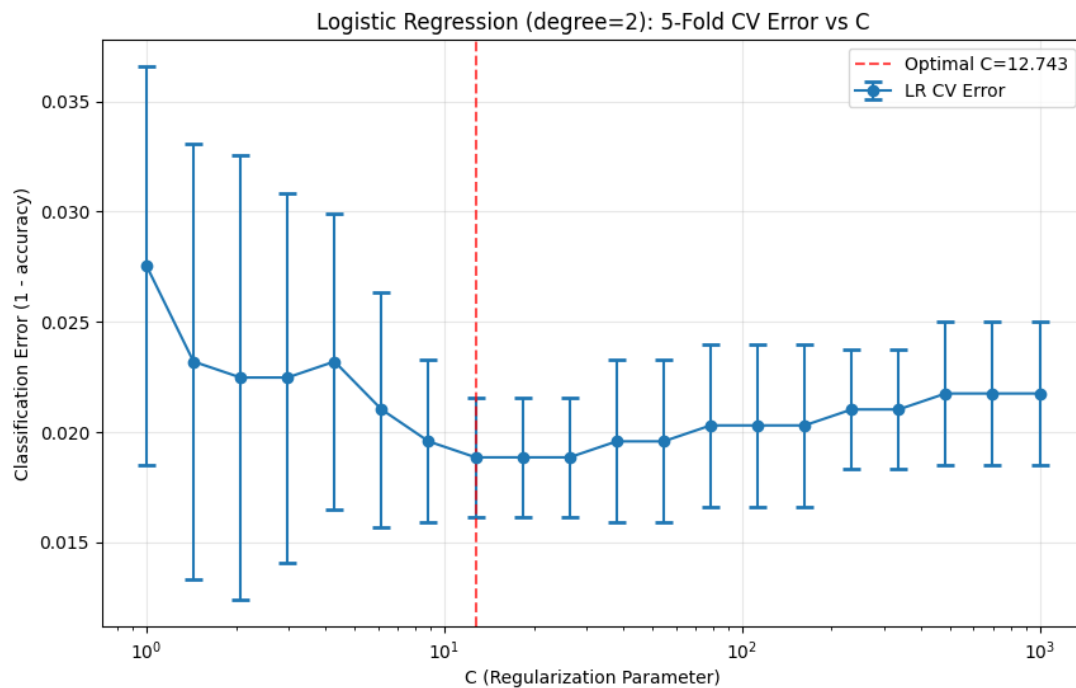
# Plot
plt.figure(figsize=(10, 6))
plt.errorbar(
    C_values, lr_cv_means, yerr=lr_cv_stds,
    marker="o", capsize=5, capthick=2, label="LR CV Error"
)
plt.xscale("log")
plt.xlabel("C (Regularization Parameter)")
plt.ylabel("Classification Error (1 - accuracy)")
plt.title(f"Logistic Regression (degree={best_degree}): 5-Fold CV Error vs C")
plt.grid(True, alpha=0.3)
plt.legend()

# mark optimal C (min error)
optimal_idx = np.argmin(lr_cv_means)
optimal_C = C_values[optimal_idx]
plt.axvline(optimal_C, color="red", linestyle="--", alpha=0.7, label=f"Optimal C = {optimal_C:.3f}")
plt.legend()
plt.show()

print(f"Optimal C value: {optimal_C:.4f}")
print(f"Minimum classification error: {lr_cv_means[optimal_idx]:.4f} ± {lr_cv_stds[optimal_idx]:.4f}")

# train final LR model again
final_lr = Pipeline(
    steps=[
        ("poly", PolynomialFeatures(degree=best_degree, include_bias=False)),
        ("logreg", LogisticRegression(penalty="l2", C=optimal_C)),
    ]
)
final_lr.fit(X, y)

```



Optimal C value: 12.7427

Minimum classification error: 0.0188 ± 0.0027

```
[10]: Pipeline(steps=[('poly', PolynomialFeatures(include_bias=False)),
                        ('logreg',
                         LogisticRegression(C=np.float64(12.742749857031335)))]])
```

```
[11]: X_fit, X_eval, y_fit, y_eval = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=42
    )

    final_lr.fit(X_fit, y_fit)

    y_pred_lr = final_lr.predict(X_eval)

    rep_lr = classification_report(
        y_eval, y_pred_lr, zero_division=0, digits=3
    )

    print(rep_lr)
```

	precision	recall	f1-score	support
-1	0.925	0.969	0.947	64
1	0.990	0.976	0.983	212

accuracy			0.975	276
macro avg	0.958	0.973	0.965	276
weighted avg	0.975	0.975	0.975	276

0.2 Train KNN

```
[ ]: # k to try
k_values = np.arange(1, 51)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

knn_cv_means = []
knn_cv_stds = []

for k in k_values:
    pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k, weights="uniform", p=2)) # p=2 Euclidean
    ])
    acc_scores = cross_val_score(pipeline, X, y, cv=cv, scoring="accuracy")
    err_scores = 1.0 - acc_scores # classification error
    knn_cv_means.append(np.mean(err_scores))
    knn_cv_stds.append(np.std(err_scores))

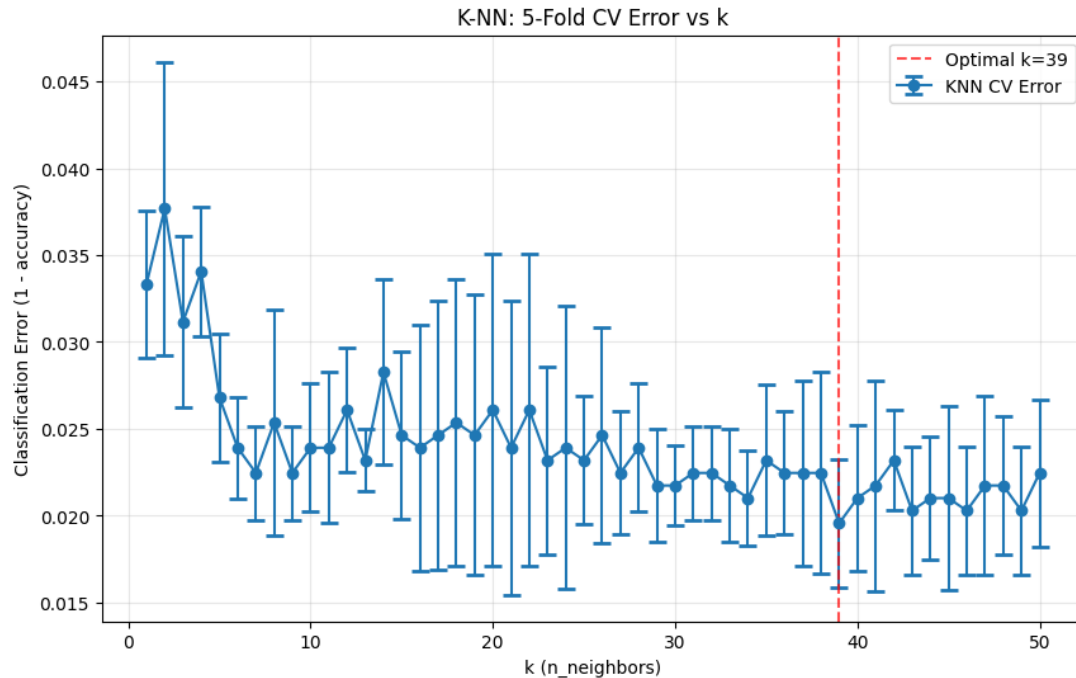
knn_cv_means = np.array(knn_cv_means)
knn_cv_stds = np.array(knn_cv_stds)

plt.figure(figsize=(10, 6))
plt.errorbar(k_values, knn_cv_means, yerr=knn_cv_stds,
             marker="o", capsize=5, capthick=2, label="KNN CV Error")
plt.xlabel("k (n_neighbors)")
plt.ylabel("Classification Error (1 - accuracy)")
plt.title("K-NN: 5-Fold CV Error vs k")
plt.grid(True, alpha=0.3)

optimal_idx = np.argmin(knn_cv_means)
optimal_k = k_values[optimal_idx]
plt.axvline(optimal_k, color="red", linestyle="--", alpha=0.7, label=f"Optimal k={optimal_k}")
plt.legend()
plt.show()

print(f"Optimal k: {optimal_k}")
```

```
print(f"Minimum classification error: {knn_cv_means[optimal_idx]:.4f} ± {knn_cv_stds[optimal_idx]:.4f}")
```



Optimal k: 39

Minimum classification error: 0.0196 ± 0.0037

```
[18]: # split the data
X_fit, X_eval, y_fit, y_eval = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

print(optimal_k)

knn = KNeighborsClassifier(n_neighbors=optimal_k, weights="uniform", p=2)

knn.fit(X_fit, y_fit)

y_pred_knn = knn.predict(X_eval)

rep_knn = classification_report(
    y_eval, y_pred_knn, zero_division=0, digits=3
)

print(rep_knn)
```

39

	precision	recall	f1-score	support
-1	0.938	0.953	0.946	64
1	0.986	0.981	0.983	212
accuracy			0.975	276
macro avg	0.962	0.967	0.965	276
weighted avg	0.975	0.975	0.975	276

1 Compare

```
[15]: print("Logistic Regression")
      print(rep_lr)
      print("\n\nK-NN")
      print(rep_knn)
```

Logistic Regression

	precision	recall	f1-score	support
-1	0.925	0.969	0.947	64
1	0.990	0.976	0.983	212
accuracy			0.975	276
macro avg	0.958	0.973	0.965	276
weighted avg	0.975	0.975	0.975	276

K-NN

	precision	recall	f1-score	support
-1	0.938	0.953	0.946	64
1	0.986	0.981	0.983	212
accuracy			0.975	276
macro avg	0.962	0.967	0.965	276
weighted avg	0.975	0.975	0.975	276

1.0.1 Regroup

```
[19]: # Predict with LR
      y_pred_lr = final_lr.predict(X)

      # Predict with KNN
```

```
y_pred_knn = knn.predict(X)
```

```
[20]: len(y_pred_knn)
```

```
[20]: 1380
```

```
[21]: len(y_pred_lr)
```

```
[21]: 1380
```

```
[22]: len(y)
```

```
[22]: 1380
```

```
[ ]: labels = [-1, 1]

# dummy
dummy = DummyClassifier(strategy="uniform", random_state=42)
dummy.fit(X, y)
y_pred_dummy = dummy.predict(X)

# cm
cm_lr = confusion_matrix(y, y_pred_lr, labels=labels)
cm_knn = confusion_matrix(y, y_pred_knn, labels=labels)
cm_dummy = confusion_matrix(y, y_pred_dummy, labels=labels)

print("Logistic Regression confusion matrix:\n", cm_lr)
print("\nK-NN confusion matrix:\n", cm_knn)
print("\nDummy confusion matrix:\n", cm_dummy)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

titlesize = 20
labelsize = 18
ticksize = 15
annotsize = 32

disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=labels)
disp_lr.plot(ax=axes[0], cmap="Blues", colorbar=False)
axes[0].set_title("Logistic Regression", fontsize=titlesize)
axes[0].tick_params(labelsize=ticksize)
axes[0].set_xlabel("Predicted label", fontsize=labelsize)
axes[0].set_ylabel("True label", fontsize=labelsize)
for txt in axes[0].texts:
    txt.set_fontsize(annotsize)
```

```

disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn,
    ↪display_labels=labels)
disp_knn.plot(ax=axes[1], cmap="Greens", colorbar=False)
axes[1].set_title("kNN", fontsize=titlesize)
axes[1].tick_params(labelsize=ticksiz)
axes[1].set_xlabel("Predicted label", fontsize=labelsize)
axes[1].set_ylabel("True label", fontsize=labelsize)
for txt in axes[1].texts:
    txt.set_fontsize(annotsize)

disp_dummy = ConfusionMatrixDisplay(confusion_matrix=cm_dummy,
    ↪display_labels=labels)
disp_dummy.plot(ax=axes[2], cmap="Reds", colorbar=False)
axes[2].set_title("Dummy (Random)", fontsize=titlesize)
axes[2].tick_params(labelsize=ticksiz)
axes[2].set_xlabel("Predicted label", fontsize=labelsize)
axes[2].set_ylabel("True label", fontsize=labelsize)
for txt in axes[2].texts:
    txt.set_fontsize(annotsize)

plt.tight_layout()
plt.show()

```

Logistic Regression confusion matrix:

```

[[ 308   11]
 [   15 1046]]

```

K-NN confusion matrix:

```

[[ 305   14]
 [   12 1049]]

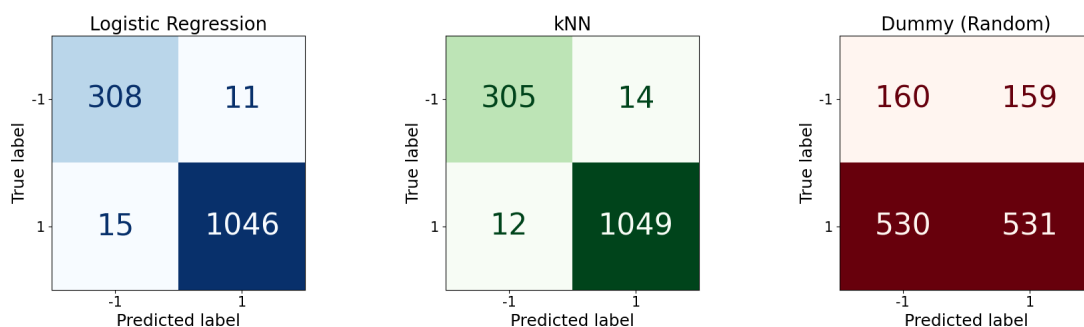
```

Dummy (Random) confusion matrix:

```

[[160 159]
 [530 531]]

```




```

[26]: y_prob_lr = final_lr.predict_proba(X)[: , 1]
      y_prob_knn = knn.predict_proba(X)[: , 1]

      dummy = DummyClassifier(strategy="uniform", random_state=42)
      dummy.fit(X, y)
      y_prob_dummy = dummy.predict_proba(X)[: , 1]

      fpr_lr, tpr_lr, _ = roc_curve(y, y_prob_lr, pos_label=1)
      fpr_knn, tpr_knn, _ = roc_curve(y, y_prob_knn, pos_label=1)
      fpr_dummy, tpr_dummy, _ = roc_curve(y, y_prob_dummy, pos_label=1)

      epsilon = 1e-4
      fpr_lr = np.maximum(fpr_lr, epsilon)
      fpr_knn = np.maximum(fpr_knn, epsilon)
      fpr_dummy = np.maximum(fpr_dummy, epsilon)

      auc_lr = roc_auc_score(y, y_prob_lr)
      auc_knn = roc_auc_score(y, y_prob_knn)
      auc_dummy = roc_auc_score(y, y_prob_dummy)

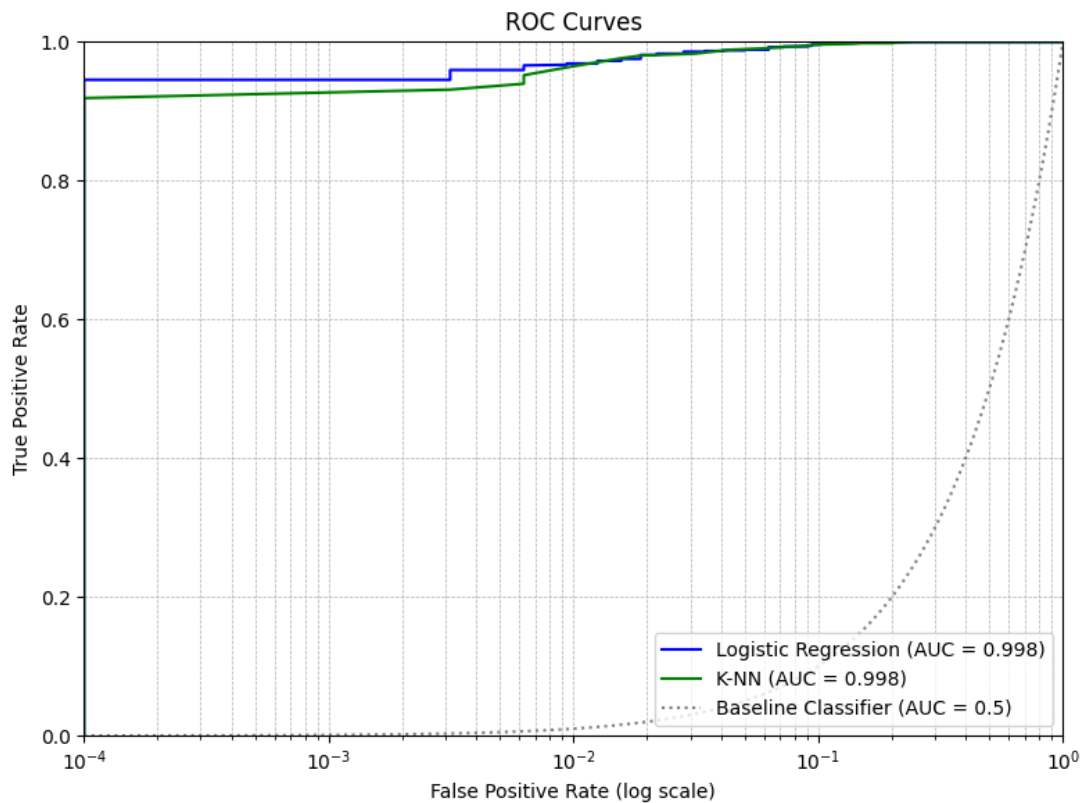
      plt.figure(figsize=(8, 6))
      plt.plot(fpr_lr, tpr_lr, color="blue", label=f"Logistic Regression (AUC = {auc_lr:.3f})")
      plt.plot(fpr_knn, tpr_knn, color="green", label=f"K-NN (AUC = {auc_knn:.3f})")

      theoretical_fpr = np.logspace(-4, 0, 200)
      plt.plot(theoretical_fpr, theoretical_fpr, color="gray", linestyle=":", label="Baseline Classifier (AUC = 0.5)")

      plt.xscale("log")
      plt.xlim([epsilon, 1])
      plt.ylim([0, 1])

      plt.title("ROC Curves")
      plt.xlabel("False Positive Rate (log scale)")
      plt.ylabel("True Positive Rate")
      plt.legend(loc="lower right")
      plt.grid(True, which="both", linestyle="--", linewidth=0.5)
      plt.tight_layout()
      plt.show()

```



best model depends on what you are looking for best TPR or best FPR

```
[29]: x_min, x_max = X[:, 0].min() - 0.1, X[:, 0].max() + 0.1
      y_min, y_max = X[:, 1].min() - 0.1, X[:, 1].max() + 0.1

      # Create mesh grid
      xx, yy = np.meshgrid(
          np.linspace(x_min, x_max, 500),
          np.linspace(y_min, y_max, 500)
      )

      # Predict class labels over the grid
      Z = final_lr.predict(np.c_[xx.ravel(), yy.ravel()])
      Z = Z.reshape(xx.shape)

      # Plot decision regions: 0=red, 1=blue
      plt.figure(figsize=(8, 6))
      cmap_bg = ListedColormap(['red', 'blue'])
      plt.contourf(xx, yy, Z, cmap=cmap_bg, alpha=0.3)

      # Overlay training data (0=red, 1=blue) with legend
```

```

scatter_cmap = ListedColormap(['blue', 'red'])

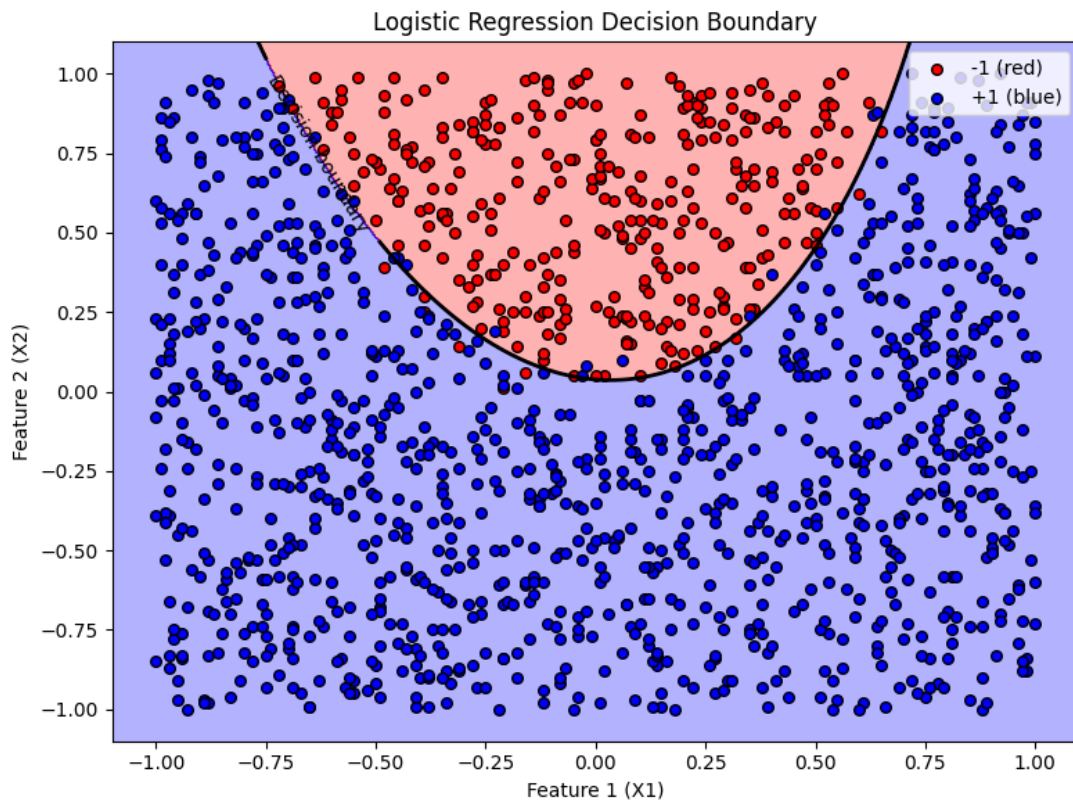
# Plot the two classes separately for legend clarity
for val, color, label in zip([-1, 1], ['red', 'blue'], ['-1 (red)', '+1 (blue)']):
    idx = (y == val)
    plt.scatter(X[idx, 0], X[idx, 1], c=color, label=label, edgecolor="k", s=30)

plt.title("Logistic Regression Decision Boundary")
plt.xlabel("Feature 1 (X1)")
plt.ylabel("Feature 2 (X2)")

# Draw the decision boundary contour (class = 0.5)
contour = plt.contour(
    xx,
    yy,
    final_lr.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1].reshape(xx.shape),
    levels=[0.5],
    colors="black",
    linewidths=2,
)
plt.clabel(contour, inline=True, fontsize=10, fmt="Decision boundary")

plt.legend(loc="upper right")
plt.tight_layout()
plt.show()

```



```
[28]: x_min, x_max = X[:, 0].min() - 0.1, X[:, 0].max() + 0.1
      y_min, y_max = X[:, 1].min() - 0.1, X[:, 1].max() + 0.1

      xx, yy = np.meshgrid(
          np.linspace(x_min, x_max, 500),
          np.linspace(y_min, y_max, 500)
      )

      Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
      Z = Z.reshape(xx.shape)

      cmap_bg = ListedColormap(['red', 'blue'])

      plt.figure(figsize=(8, 6))
      plt.contourf(xx, yy, Z, cmap=cmap_bg, alpha=0.3)

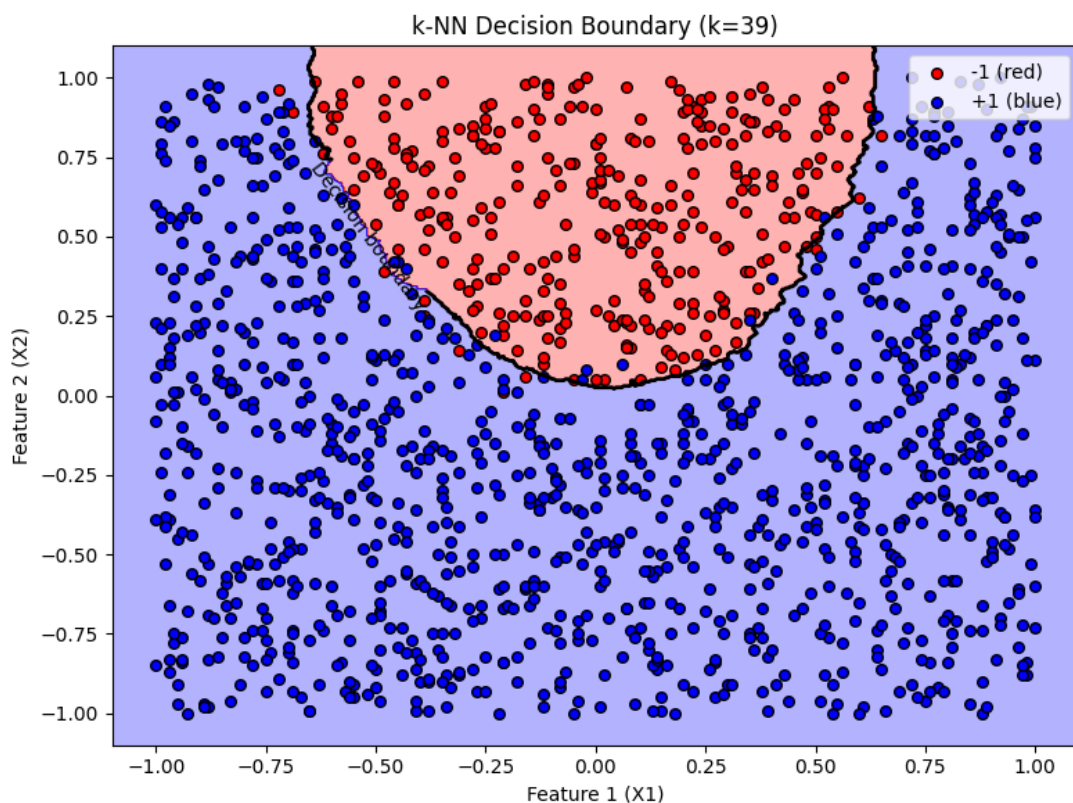
      scatter_cmap = ListedColormap(['red', 'blue'])
      for val, color, label in zip([-1, 1], ['red', 'blue'], ['-1 (red)', '+1 (blue)']):
          idx = (y == val)
```

```
plt.scatter(X[idx, 0], X[idx, 1], c=color, label=label, edgecolor="k", s=30)

plt.title(f"k-NN Decision Boundary (k={knn.n_neighbors})")
plt.xlabel("Feature 1 (X1)")
plt.ylabel("Feature 2 (X2)")

contour = plt.contour(xx, yy, Z, levels=[0.5], colors="black", linewidths=2)
plt.clabel(contour, inline=True, fontsize=10, fmt="Decision boundary")

plt.legend(loc="upper right")
plt.tight_layout()
plt.show()
```



[]: